S35 - Evidencia de aprendizaje 2. Optimizando la productividad en el mundo del software

Integrante:

Raúl Santiago Morales Henao

Institución Universitaria Digital de Antioquia (IU Digital)

Programación para Análisis de Datos - PREICA2602B020048

Docente: Andrés Felipe Palacio

6 de noviembre del 2026

1. Introducción

En el desarrollo de software moderno y la ingeniería de análisis de datos, la construcción de un algoritmo funcional de extracción web (Web Scraping) constituye apenas la primera fase del ciclo de vida de una solución tecnológica. Cuando los proyectos escalan hacia entornos colaborativos y de producción, surgen desafíos críticos relacionados con la trazabilidad de los cambios en el código fuente, el trabajo paralelo entre múltiples desarrolladores, la detección temprana de fallos ante modificaciones en la estructura HTML de los sitios web y la publicación oportuna de los datos actualizados.

El presente informe documenta el desarrollo de la Evidencia de Aprendizaje 2 correspondiente a la Unidad 2 ('Optimizando la productividad en el mundo del software'), dando continuidad directa al estudio iniciado en la Unidad 1 sobre el monitoreo de precios del Apple iPhone 17 Pro Max en la plataforma Mercado Libre Colombia. En esta segunda etapa, el proyecto evoluciona desde un script de extracción local hacia una arquitectura integral gobernada por control de versiones distribuido con Git y GitHub, acompañada de un flujo de trabajo DevOps de Integración Continua y Despliegue Continuo (CI/CD) mediante GitHub Actions.

A lo largo del documento se retoma el contexto de la plataforma y el producto analizado, se define el tema de interés enfocado en la automatización y productividad de software, se formulan los objetivos bajo el escenario de un equipo de desarrollo colaborativo, se explica la metodología de scraping refinada junto con el flujo de ramas, commits, fusiones y Pull Requests en GitHub, y se detallan los resultados de las pruebas automatizadas y del despliegue continuo en la nube a través de GitHub Pages.

2. Descripción de la Página y Artículo Analizado

2.1. Plataforma de Comercio Electrónico: Mercado Libre Colombia

Mercado Libre (https://www.mercadolibre.com.co) es la plataforma líder de comercio electrónico y servicios financieros digitales en América Latina. Su catálogo de búsqueda para telefonía móvil de alta gama (https://listado.mercadolibre.com.co/iphone-17-pro-max) reúne en una misma interfaz a la tienda oficial de Apple, distribuidores autorizados, importadores directos y vendedores particulares, generando un mercado altamente dinámico donde los precios, descuentos porcentuales y ganadores del catálogo ('Buy Box') se actualizan de manera constante.

Desde el punto de vista técnico, Mercado Libre utiliza la arquitectura frontend 'Nordic Framework', la cual estructura cada resultado de búsqueda mediante tarjetas modulares con la clase CSS '.poly-card' dentro de nodos 'li.ui-search-layout__item'. En cada tarjeta coexisten múltiples contenedores monetarios ('.andes-money-amount__fraction'): el precio anterior tachado ('s.andes-money-amount--previous'), el precio actual real de venta ('div.poly-price__current') y el valor de las cuotas mensuales ('span.poly-price__installments'). Asimismo, la plataforma implementa controles de seguridad anti-automatización que redirigen peticiones HTTP sin sesión autenticada hacia el módulo de verificación 'gz/account-verification'.

2.2. Artículo Seleccionado: Apple iPhone 17 Pro Max

El producto objeto de seguimiento continúa siendo el Apple iPhone 17 Pro Max, buque insignia de Apple equipado con pantalla OLED Super Retina XDR de 6.9 pulgadas con tecnología ProMotion a 120 Hz, procesador A19 Pro con enfriamiento por evaporación, chasis unibody de aluminio forjado con protección Ceramic Shield 2, sistema de triple cámara trasera Pro de 48 MP con zoom óptico 8x y configuraciones de almacenamiento de 256 GB, 512 GB y 1 TB en acabados Naranja Cósmico, Azul Profundo y Plata.

La elección de este dispositivo como caso de estudio para la implementación del pipeline DevOps resulta ideal debido a la alta volatilidad tarifaria entre las 48 publicaciones analizadas y a la necesidad de contar con pruebas unitarias automatizadas que garanticen que el scraper extraiga siempre el precio real vigente con descuento y no el precio de lista tachado.

3. Descripción del Tema de Interés en la Unidad 2

El tema de interés desarrollado en esta segunda entrega es la Automatización del Ciclo de Vida del Pipeline de Extracción de Datos mediante Prácticas DevOps, Control de Versiones en GitHub e Integración y Despliegue Continuo (CI/CD). Mientras que en la Unidad 1 el enfoque se centró en la captura y limpieza de las 48 publicaciones del iPhone 17 Pro Max, en la Unidad 2 el objetivo es transformar dicho extractor en un producto de software mantenible, auditable, colaborativo y desplegado en la nube bajo cuatro pilares estratégicos:

- Control de Versiones Distribuido y Trabajo en Equipo (Git y GitHub): Organización del código fuente en un repositorio público (https://github.com/shenao356/mercadolibre-iphone17pm-shenao) aplicando una estrategia de ramificación ('main', 'develop', 'rama-scraper', 'rama-pruebas' y 'rama-automatizacion') con commits escalonados y descriptivos.

- Aseguramiento Automatizado de Calidad (CI Quality Gate): Implementación de una suite de 6 pruebas unitarias y de integración con 'unittest' ('tests/test_scraper.py') que verifican la precisión del selector CSS ('div.poly-price__current' frente a precios tachados '<s>'), la normalización de capacidades mediante expresiones regulares y la integridad estructural de los archivos CSV y JSON.

- Corrección Evolutiva y Revisión de Código mediante Pull Requests: Uso de solicitudes de extracción (Pull Request #1 fusionado en GitHub) para auditar e integrar mejoras

en el código, incluyendo la eliminación de rutas locales dependientes del sistema operativo y la captura exacta de descuentos activos (18% OFF, 42% OFF y 21% OFF).

- Despliegue Continuo en la Nube (Continuous Deployment - CD): Generación automatizada de un tablero web analítico de alto contraste ('docs/index.html') y una API estática JSON ('docs/api/iphone17_data.json') desplegados públicamente mediante GitHub Actions y GitHub Pages (https://shenao356.github.io/mercadolibre-iphone17pm-shenao/).

4. Objetivos y Métricas de Éxito

Contextualización del Caso de Equipo de Desarrollo: Como integrante de un equipo de ingeniería de software y datos encargado del sistema de monitoreo comercial del iPhone 17 Pro Max en Mercado Libre Colombia, se requiere estructurar el proyecto bajo control de versiones con Git y GitHub y establecer un pipeline de CI/CD con GitHub Actions que ejecute pruebas automatizadas y despliegue continuo en la nube.

4.1. Objetivo General

Implementar una arquitectura de ingeniería de software colaborativa basada en la metodología DevOps, utilizando Git y GitHub para la gestión del control de versiones por ramas y Pull Requests, e integrando un pipeline automatizado de CI/CD con GitHub Actions que valide mediante pruebas unitarias y despliegue de forma continua el sistema de extracción y visualización de precios del iPhone 17 Pro Max en Mercado Libre Colombia.

4.2. Objetivos Específicos

- Estructurar un repositorio en GitHub (https://github.com/shenao356/mercadolibre-iphone17pm-shenao) aplicando un modelo de ramificación ('main', 'develop' y ramas de funcionalidades) con un historial de commits escalonados, claros y trazables.

- Refinar el módulo de extracción web ('scraper_mercadolibre.py') para que sea 100% compatible con entornos en la nube (multiplataforma) y seleccione con precisión el precio actual de venta ('div.poly-price__current') diferenciándolo del precio anterior tachado ('s.andes-money-amount--previous').

- Desarrollar una batería de pruebas automatizadas con 'unittest' ('tests/test_scraper.py') que actúe como compuerta de calidad (Quality Gate), validando el parseo del DOM, la distinción entre precios con descuento y precios tachados, y la integridad de los 48 registros en CSV y JSON.

- Configurar y ejecutar un flujo de integración y despliegue continuo (CI/CD) mediante GitHub Actions y GitHub Pages que publique automáticamente el tablero analítico interactivo y el servicio de datos JSON tras cada actualización aprobada en la rama principal.

4.3. Métricas de Éxito del Proyecto DevOps

De acuerdo con los lineamientos y la rúbrica de evaluación de la Unidad 2, el desempeño técnico de la solución se evalúa a través de las siguientes métricas verificables:

Métrica de Éxito DevOps	**Meta Establecida**	**Resultado Obtenido en la Práctica**
Gestión de Ramas y Commits	>= 3 ramas y commits descriptivos	5 ramas activas ('main', 'develop', 'rama-scraper', 'rama-pruebas', 'rama-automatizacion') y 10 commits escalonados.
Revisión e Integración (PR)	>= 1 Pull Request fusionado	Pull Request #1 creado, documentado y fusionado

		exitosamente hacia la rama 'main' en GitHub.
Cobertura de Pruebas (CI)	100% de tests unitarios aprobados	6 de 6 pruebas automatizadas superadas ('OK' en 0.074 s) validando HTML, precios reales, CSV y JSON.
Precisión de Selector de Precio	0 errores por precios tachados	100% de precisión extrayendo 'div.poly-price__current' e identificando descuentos (18%, 42% y 21% OFF).
Despliegue Continuo (CD)	Aplicación web y API en línea	Despliegue exitoso ('status: success') en GitHub Pages con tablero web y endpoint JSON activos.

5. Metodología Empleada de Scraping y Flujo de Trabajo DevOps

5.1. Metodología Empleada de Web Scraping y Refinamiento del Selector DOM

Siguiendo la base técnica establecida en la Unidad 1, el proyecto contempla las tres metodologías fundamentales de extracción de datos en Python (BeautifulSoup, Selenium WebDriver y Scrapy) dentro del módulo 'scraper_mercadolibre.py', seleccionando BeautifulSoup ('bs4') junto con 'pandas' y expresiones regulares ('re') como motor principal de

procesamiento para el pipeline de Integración Continua debido a su bajo consumo de memoria, alta velocidad de ejecución en contenedores Linux de GitHub Actions y determinismo en las pruebas.

• BeautifulSoup (Motor Principal en CI/CD): Analiza el árbol DOM de las tarjetas '.poly-card' de Mercado Libre. Durante el ciclo de mejora continua de la Unidad 2 se detectó que, en los productos en promoción, la primera etiqueta '.andes-money-amount__fraction' dentro de la tarjeta corresponde al precio anterior tachado ('<s class="andes-money-amount--previous">'). Por ello, se refactorizó la función 'parse_mercadolibre_with_bs4()' para apuntar específicamente al nodo contenedor ' ', capturando de forma exacta el precio vigente de venta, el precio anterior tachado y el porcentaje de descuento ('18% OFF', '42% OFF', '21% OFF'), además de preservar el identificador 'wid=' de la oferta en el enlace.

• Selenium WebDriver (Módulo de Navegación Dinámica): Implementado en la función 'scrape_with_selenium(headless=True)' utilizando Microsoft Edge Chromium con banderas de mitigación de detección ('--disable-blink-features=AutomationControlled') para escenarios que requieren interacción directa con el navegador o renderizado JavaScript en vivo.

• Scrapy Framework (Arquitectura Asíncrona Escalable): Implementado en la clase 'MercadoLibreIphoneSpider(scrapy.Spider)' para demostrar el diseño de rastreadores asíncronos capaces de recorrer múltiples páginas de catálogo mediante selectores CSS encadenados.

Criterio Técnico	**BeautifulSoup (bs4)**	**Selenium WebDriver**	**Scrapy Framework**
Rol en el Pipeline CI/CD	Motor principal de parseo y pruebas	Módulo de captura dinámica en	Arquitectura de referencia para

	unitarias en GitHub Actions	navegador local/headless	escalamiento masivo multipágina
Selector de Precio Actual	Búsqueda focalizada en 'div.poly-price__current'	Localización mediante 'By.CSS_SELECTOR' sobre '.poly-card'	Selector CSS encadenado 'div.poly-price__current ::text'
Consumo de Recursos en Nube	Mínimo (~20 MB RAM, ejecución < 0.1 s)	Alto (~500 MB RAM, requiere binario WebDriver)	Moderado (~60 MB RAM, motor asíncrono Twisted)
Adaptabilidad al Cambio (DevOps)	Validado automáticamente por 6 pruebas unitarias en cada push	Requiere sesión autenticada ante bloqueo 'account-verification'	Altamente modular mediante middlewares y pipelines

5.2. Gestión de Versiones con Git y GitHub (Ramas, Fusiones y Pull Requests)

Para garantizar un desarrollo ordenado, colaborativo y libre de conflictos en el código fuente, se implementó una estrategia de ramificación inspirada en Git Flow dentro del repositorio oficial https://github.com/shenao356/mercadolibre-iphone17pm-shenao. En lugar de trabajar directamente sobre la rama de producción, el trabajo se dividió en cinco ramas especializadas con responsabilidades claramente delimitadas:

- Rama 'main' (Producción): Contiene únicamente versiones estables, probadas y listas para el despliegue público en GitHub Pages.

- Rama 'develop' (Integración): Rama troncal de desarrollo donde convergen las funcionalidades antes de pasar a producción.

- Rama 'rama-scraper' (Feature Branch 1): Dedicada a la construcción y refactorización del script 'scraper_mercadolibre.py', la eliminación de rutas locales de Windows para garantizar compatibilidad en la nube y la exportación de los datasets 'iphone_17_pro_max_mercadolibre.csv' y '.json'.

- Rama 'rama-pruebas' (Feature Branch 2): Dedicada al desarrollo de la suite de pruebas automatizadas en 'tests/test_scraper.py' para validar la extracción del precio real frente al precio tachado y la completitud de los 48 registros.

- Rama 'rama-automatizacion' (Feature Branch 3): Dedicada a la creación del generador de despliegue continuo 'deploy_dashboard.py', los artefactos web en 'docs/index.html', el flujo de GitHub Actions 'workflows/ci_cd_pipeline.yml' y el documento escrito en PDF 'Morales_Henao_Raul_Santiago_Optimizando_EA2.pdf'.

El flujo de integración se ejecutó de manera escalonada: cada rama de funcionalidad ('rama-scraper', 'rama-pruebas' y 'rama-automatizacion') se fusionó secuencialmente hacia 'develop' mediante comandos 'git merge', verificando que las pruebas unitarias pasaran sin errores. Posteriormente, se abrió y fusionó el Pull Request #1 (https://github.com/shenao356/mercadolibre-iphone17pm-shenao/pull/1) para integrar los cambios validados hacia la rama 'main', manteniendo un historial limpio de 10 commits con mensajes claros y naturales.

Paso /	**Rama de Trabajo**	**Mensaje del Commit en Git**	**Artefactos Integrados**

Orden			
Paso 1	main	Subo archivos base de la actividad 1 y datos del iPhone 17 Pro Max	.gitignore, requirements.txt, CSV y JSON
Paso 2	rama-scraper	Agrego el script de python para hacer el web scraping en Mercado Libre	scraper_mercadolibre.py (multiplataforma)
Paso 3	develop	Uno la rama del scraper con la rama develop	Fusión de 'rama-scraper' hacia 'develop'
Paso 4	rama-pruebas	Creo las pruebas automaticas para revisar que los precios y datos esten bien	tests/test_scraper.py (suite unittest)
Paso 5	develop	Uno la rama de pruebas	Fusión de 'rama-pruebas' hacia 'develop'

		automaticas con develop	
Paso 6	rama-automatizacion	Configuro la automatizacion del proyecto y el reporte web en la carpeta docs	deploy_dashboard.py, docs/, workflows/, README.md
Paso 7	develop	Uno la rama de automatizacion y reporte web con develop	Fusión de 'rama-automatizacion' hacia 'develop'
Paso 8	develop	Agrego el documento en PDF de la actividad 2 con las pruebas y resultados	Morales_Henao_Raul_Santiago_Optimizando_EA2.pdf
Paso 9	main (Merge PR #1)	Merge pull request #1 from shenao356/develop	Integración de 'develop' a producción ('main')
Paso 10	main / develop	Mejoro los colores y el	Rediseño visual de alto contraste en docs/index.html

		contraste del tablero web para que se vea mas claro	

5.3. Explicación Detallada del Pipeline CI/CD con GitHub Actions y Despliegue Continuo

El flujo de trabajo DevOps implementado automatiza de extremo a extremo la validación y publicación del proyecto cada vez que se realiza un 'git push' o se fusiona un Pull Request hacia la rama 'main'. La arquitectura del pipeline se divide en dos fases secuenciales interdependientes documentadas en 'workflows/ci_cd_pipeline.yml' y ejecutadas en los servidores de GitHub Actions:

• Etapa 1 - Integración Continua y Compuerta de Calidad (Job 'ci-quality-gate'): Aprovisiona una máquina virtual 'ubuntu-latest', descarga el código del repositorio ('actions/checkout@v4'), configura Python 3.11 ('actions/setup-python@v5') e instala las dependencias ('beautifulsoup4', 'pandas', 'lxml'). A continuación, ejecuta 'python -m unittest discover -s tests -v' y 'python scraper_mercadolibre.py'. Si alguna de las 6 pruebas unitarias falla (por ejemplo, si el scraper tomara por error un precio tachado o si el CSV tuviera valores nulos), el pipeline se detiene inmediatamente e impide el despliegue.

• Etapa 2 - Despliegue Continuo en la Nube (Job 'cd-cloud-deployment' / 'pages build and deployment'): Una vez superadas las pruebas, el script 'deploy_dashboard.py' transforma los 48 registros validados en dos artefactos de producción dentro de la carpeta 'docs/': (1) un tablero web interactivo ('docs/index.html') con tarjetas de KPIs estadísticos, resumen por capacidad de almacenamiento y tabla completa con indicadores de descuento y enlaces

directos a Mercado Libre, y (2) un endpoint API JSON estático ('docs/api/iphone17_data.json'). Finalmente, el workflow oficial 'pages build and deployment' de GitHub Actions empaqueta y publica el sitio en GitHub Pages (https://shenao356.github.io/mercadolibre-iphone17pm-shenao/).

Fase del Pipeline	**Herramienta / Acción**	**Comando o Proceso Ejecutado**	**Resultado Verificado**
1. Aprovisionamiento	GitHub Actions Runner	Checkout del código e instalación de Python y librerías ('requirements.txt')	Entorno limpio y reproducible en la nube sin rutas locales.
2. Pruebas Unitarias (CI)	Python unittest	python -m unittest discover -s tests -v (6 pruebas automatizadas)	6/6 tests aprobados en 0.074 s (0 fallos, 0 errores).
3. Construcción (Build)	deploy_dashboard.py	Generación de 'docs/index.html' y 'docs/api/iphone17_data.json'	Tablero web actualizado con KPIs y precios reales de venta.
4. Despliegue Cloud (CD)	GitHub Pages Action	Workflow 'pages build and deployment' (build -> report-build-status -> deploy)	Sitio web y API JSON publicados en

			vivo con estado 'success'.

6. Resultados y Análisis de Datos

La implementación del ciclo DevOps en la Unidad 2 no solo permitió automatizar el despliegue del proyecto en GitHub, sino que además mejoró sustancialmente la calidad de los datos extraídos respecto a la Unidad 1 gracias a la detección y corrección del selector de precios en artículos con descuento activo.

6.1. Resultados de la Suite de Pruebas Automatizadas (Quality Gate)

La suite 'tests/test_scraper.py' evalúa seis dimensiones críticas del sistema antes de autorizar cualquier publicación en producción. En la siguiente tabla se detallan los resultados obtenidos durante la ejecución automatizada:

ID Prueba Unitaria	**Descripción de la Validación Técnica**	**Estado**	**Impacto en la Calidad del Dato**
test_01_html_extraction_count	Verifica que BeautifulSoup extraiga la totalidad de tarjetas '.poly-card' del DOM de prueba.	OK (Pass)	Previene pérdida silenciosa de productos ante cambios en la grilla HTML.

test_02_current_price_vs_crossed_out	Valida que se extraiga el precio actual ('5.685.686') de 'div.poly-price__current' y no el tachado ('6.999.000').	OK (Pass)	Elimina el sesgo de sobreestimación causado por precios 'Antes' tachados.
test_03_storage_capacity_normalization	Comprueba la clasificación por expresiones regulares en 256GB, 512GB y 1TB.	OK (Pass)	Asegura la segmentación correcta para el análisis comparativo de capacidades.
test_04_shipping_and_wid_url	Verifica la normalización de 'Envío Gratis' y la conservación del parámetro 'wid=' en la URL.	OK (Pass)	Mantiene la trazabilidad hacia la oferta específica dentro del catálogo.
test_05_production_csv_integrity	Audita que el archivo CSV	OK (Pass)	Garantiza cero corrupción de

	contenga 48 registros sin nulos y precios en rango válido.		datos en el archivo tabular de producción.
test_06_production_json_integrity	Audita la validez sintáctica y estructural del archivo JSON exportado.	OK (Pass)	Asegura la interoperabilidad de la API estática publicada en GitHub Pages.

6.2. Resumen Estadístico Descriptivo de Precios Reales de Venta (COP)

Al refinar el selector de BeautifulSoup para capturar exclusivamente el precio actual de venta ('div.poly-price__current') e ignorar los precios anteriores tachados en las publicaciones con descuento (como el artículo #1 con 18% OFF, el #3 con 42% OFF y el #7 con 21% OFF), el precio promedio del mercado disminuyó de $ 4.917.059 COP a $ 4.776.156 COP, reflejando con total fidelidad el costo real que paga el comprador:

Parámetro Estadístico	**Valor Numérico (COP)**	**Interpretación Analítica y de Negocio**
Muestra Analizada (N)	48 artículos	Total de publicaciones validadas y desplegadas en el tablero web y API JSON.

Precio Promedio (Media)	$ 4.776.156	Costo medio real de venta tras descontar los precios de lista tachados.
Precio Mediano (Mediana)	$ 4.600.000	El 50% de las ofertas se ubica por debajo de $ 4.600.000 COP.
Desviación Estándar (s)	$ 1.129.410	Dispersión moderada-alta explicada por la diferencia entre nuevos y reacondicionados.
Coeficiente de Variación (CV)	23.65%	Heterogeneidad tarifaria reducida frente al 27.62% previo a la corrección del selector.
Precio Mínimo	$ 2.667.777	Oferta de entrada correspondiente a equipo reacondicionado / importación.
Precio Máximo	$ 7.990.000	Tope superior en publicación sin descuento activo para entrega inmediata.

Rango Total (Amplitud)	$ 5.322.223	Brecha monetaria entre la publicación más económica y la más costosa.

6.3. Distribución y Comparativa de Precios por Capacidad de Almacenamiento

La segmentación automatizada por capacidad de almacenamiento interno refleja el predominio de la versión de 256 GB y el ajuste en el promedio del segmento de 512 GB una vez aplicado el descuento del 42% OFF en el catálogo principal:

Capacidad	**Publicaciones**	**% Muestra**	**Precio Promedio**	**Precio Mínimo**	**Precio Máximo**
256 GB	35	72.9%	$ 4.603.335	$ 3.500.000	$ 7.990.000
512 GB	8	16.7%	$ 5.223.648	$ 2.667.777	$ 7.550.000
1 TB	1	2.1%	$ 5.990.098	$ 5.990.098	$ 5.990.098
No especificada	4	8.3%	$ 5.211.975	$ 3.999.000	$ 7.000.000

6.4. Muestra Representativa de Artículos Desplegados en el Tablero Web

A continuación, se presenta la muestra de los primeros 8 registros desplegados en el tablero interactivo (https://shenao356.github.io/mercadolibre-iphone17pm-shenao/), evidenciando la captura simultánea del precio actual de venta, el precio anterior tachado y la etiqueta de descuento:

#	Título del Producto Extraído	Precio Actual (COP)	Precio Anterior / Descuento	Capacidad
1	Apple iPhone 17 Pro Max (256 GB) - Naranja cósmico - Distribuidor Autorizado	$ 5.685.686	Antes: $ 6.999.000 (18% OFF)	256GB
2	iPhone 17 Pro Max 256gb Nuevo | Silver Y Deep Blue	$ 4.649.000	Sin descuento tachado	256GB
3	Apple iPhone 17 Pro Max (512 GB) - Azul profundo	$ 5.699.900	Antes: $ 9.999.000 (42% OFF)	512GB
4	Apple iPhone 17 Pro Max (1 TB) - Color plata	$ 5.990.098	Sin descuento tachado	1TB
5	Apple iPhone 17 Pro Max 256gb	$ 4.999.000	Sin descuento tachado	256GB

	Naranja Sim Física			
6	iPhone 17 Pro Max Tres Colores	$ 4.998.900	Sin descuento tachado	No especificada
7	Smartphone Esim Azul Apple iPhone 17 Pro Max 256 Gb	$ 4.329.639	Antes: $ 5.480.559 (21% OFF)	256GB
8	Apple iPhone 17 Pro Max (256 GB) - Naranja cósmico (Nuevo con caja abierta)	$ 4.700.000	Sin descuento tachado	256GB

7. Conclusiones

7.1. Conclusiones sobre Control de Versiones, Automatización y Productividad DevOps

1. Eficiencia y Trazabilidad con Git y GitHub: La estructuración del proyecto en cinco ramas ('main', 'develop', 'rama-scraper', 'rama-pruebas' y 'rama-automatizacion') permitió aislar el desarrollo de nuevas funcionalidades sin comprometer la estabilidad de la rama principal. El uso de Pull Requests (PR #1) demostró ser un mecanismo indispensable para documentar y auditar cambios antes de su paso a producción.

2. Valor de las Pruebas Automatizadas en Web Scraping: Los sitios de comercio electrónico presentan estructuras DOM complejas donde conviven precios tachados, precios con descuento y valores de cuotas. La creación de la prueba unitaria 'test_02_current_price_vs_crossed_out_price' en 'tests/test_scraper.py' garantizó de forma automática que el scraper seleccione siempre 'div.poly-price__current', evitando distorsiones estadísticas en el análisis.

3. Despliegue Continuo Accesible y Reproducible: La eliminación de rutas locales absolutas y la integración de 'deploy_dashboard.py' con GitHub Actions y GitHub Pages permitieron transformar un conjunto de archivos planos en una aplicación web de alto contraste y una API JSON pública que se actualizan automáticamente en segundos tras cada 'git push'.

7.2. Conclusiones sobre el Monitoreo Comercial en Mercado Libre Colombia

1. Impacto Real de los Descuentos en el Mercado: Al separar correctamente el precio de venta vigente del precio anterior tachado, se comprobó que las promociones en distribuidores autorizados y tiendas destacadas (de hasta el 42% OFF) reducen el promedio general del mercado en más de $ 140.000 COP por unidad.

2. Dinámica de Catálogos ('/p/MCO...') frente a Publicaciones Directas ('articulo.mercadolibre.com.co'): Se identificó que las páginas de catálogo de Mercado Libre rotan dinámicamente al vendedor ganador ('Buy Box') y actualizan sus precios día a día según el stock disponible, mientras que las publicaciones individuales mantienen precios estables a lo largo del tiempo, lo que refuerza la importancia de contar con un pipeline automatizado de actualización periódica.

8. Referencias Bibliográficas (Normas APA 7ª Edición)

Apple Inc. (2026). iPhone: Especificaciones técnicas y modelos comerciales. Apple Latinoamérica. https://www.apple.com/co/iphone/

Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress. https://git-scm.com/book/es/v2

Forsgren, N., Humble, J., & Kim, G. (2018). Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations. IT Revolution Press.

GitHub Inc. (2026). GitHub Actions Documentation: Automate your workflow from idea to production. https://docs.github.com/es/actions

Humble, J., & Farley, D. (2010). Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Addison-Wesley Professional.

Kim, G., Humble, J., Debois, P., Willis, J., & Forsgren, N. (2021). The DevOps Handbook: How to Create World-Class Agility, Reliability, & Security in Technology Organizations (2nd ed.). IT Revolution Press.

Mercado Libre Colombia. (2026). Catálogo de publicaciones electrónicas: iPhone 17 Pro Max. https://listado.mercadolibre.com.co/iphone-17-pro-max

Mitchell, R. (2018). Web Scraping with Python: Collecting More Data from the Modern Web (2nd ed.). O'Reilly Media.

Python Software Foundation. (2026). unittest — Unit testing framework (Python 3.11 Documentation). https://docs.python.org/3/library/unittest.html

Richardson, L. (2023). Beautiful Soup Documentation. Crummy Software. https://www.crummy.com/software/BeautifulSoup/bs4/doc/

9. Anexo: Código Fuente Documentado de la Solución

A continuación, se adjunta el código fuente principal implementado y desplegado en el repositorio oficial de GitHub (https://github.com/shenao356/mercadolibre-iphone17pm-shenao), incluyendo el extractor con BeautifulSoup, la suite de pruebas unitarias automatizadas y la configuración del pipeline CI/CD.

```
        #
================================================================================
===
        # 1. EXTRACTOR BEAUTIFULSOUP MULTIPLATAFORMA (scraper_mercadolibre.py)
        #
================================================================================
===
        import os, re, json, pandas as pd
        from bs4 import BeautifulSoup


        def parse_mercadolibre_with_bs4(html_content):
            soup = BeautifulSoup(html_content, 'html.parser')
            items = soup.find_all('li', class_=re.compile(r'ui-search-layout__item'))
            extracted_data = []
            for item in items:
                title_tag = item.find(class_=re.compile(r'poly-component__title|ui-search-
item__title'))
```

```
            # Extraer el precio actual real desde div.poly-price__current (ignorando el tachado
<s>)
            current_price_box = item.find('div', class_=re.compile(r'poly-price__current'))
            if current_price_box:
                price_tag = current_price_box.find(class_=re.compile(r'andes-money-
amount__fraction'))
                discount_tag = current_price_box.find(class_=re.compile(r'poly-
price__discount|polylabel-pill'))
            else:
                price_tag = item.find(class_=re.compile(r'andes-money-amount__fraction'))
                discount_tag = None
            prev_box = item.find('s', class_=re.compile(r'andes-money-amount--previous'))
            prev_price_tag = prev_box.find(class_=re.compile(r'andes-money-
amount__fraction')) if prev_box else None
            link_tag = item.find('a', href=True)
            if title_tag and price_tag:
                title = title_tag.get_text(strip=True)
                price_clean = int(price_tag.get_text(strip=True).replace('.', '').replace('$', '').strip())
                cap_match = re.search(r'(256\s*GB|512\s*GB|1\s*TB)', title, re.IGNORECASE)
                capacity = cap_match.group(1).upper().replace(' ', '') if cap_match else 'No
especificada'
```

```
                extracted_data.append({
                    'id': len(extracted_data) + 1,
                    'titulo': title,
                    'precio_cop': price_clean,
                    'precio_formateado': f'$ {price_clean:,.0f}'.replace(',', '.'),
                    'precio_anterior': f"$ {prev_price_tag.get_text(strip=True)}" if prev_price_tag
else '-',
                    'descuento': discount_tag.get_text(strip=True) if discount_tag else '-',
                    'capacidad': capacity,
                    'enlace': link_tag['href'] if link_tag else 'N/A'
                })
        return extracted_data


    #
================================================================================
===
    # 2. SUITE DE PRUEBAS UNITARIAS AUTOMATIZADAS (tests/test_scraper.py)
    #
================================================================================
===
    import unittest
```

```
class TestMercadoLibreScraperPipeline(unittest.TestCase):
    def test_current_price_vs_crossed_out_price(self):
        html = '''<li class="ui-search-layout__item">
            <a class="poly-component__title"
href="https://mercadolibre.com.co/p/MCO1">iPhone 17 Pro Max 256 GB</a>
            <s class="andes-money-amount--previous"><span class="andes-money-
amount__fraction">6.999.000</span></s>
            <div class="poly-price__current">
                <span class="andes-money-amount__fraction">5.685.686</span>
                <span class="polylabel-pill">18% OFF</span>
            </div></li>'''
        data = parse_mercadolibre_with_bs4(html)
        self.assertEqual(data[0]['precio_cop'], 5685686)
        self.assertEqual(data[0]['precio_anterior'], '$ 6.999.000')
        self.assertEqual(data[0]['descuento'], '18% OFF')
```